StateGuard: Failure Propagation and State-Consistent Recovery in Tool-Using Multi-Agent Workflows

Xinying Liu Jingwen Zhang Haodi Yuan
University of California San Diego, CSE 291A — Option 5
xil277@ucsd.edu jiz333@ucsd.edu hay101@ucsd.edu

Abstract. StateGuard investigates failure propagation and selective recovery under changing shared state. We implement a versioned deployment runtime and record 99 real-model clean/fault pairs on AWS. Across 30 combinations valid for all policies, completion is 30% without recovery, 100% with full restart, and 80% with verification. Restart approximately doubles tokens and latency. We report Phase 1 evidence and propose selective-recovery evaluation.

1 Research question and workflow

An incorrect port can propagate from analysis into a plan and committed configuration. Replanning leaves the write behind; snapshot rollback may erase a later administrator update. Our questions are: **RQ1**, which message and state dependencies carry failures; **RQ2**, which work remains reusable after legitimate updates; and **RQ3**, when does selective recovery save more than its bookkeeping costs?

A representative trace begins with an obsolete guide claiming that the API can use database port 3306. The Dependency result passes this claim to the Planner, which writes the resulting API configuration. Meanwhile, a worker scales independently and an administrator raises API replicas. Correct recovery must repair the port while preserving both valid changes, provided the resulting CPU budget still holds. This trace makes consumption, committed effects, and intervening state changes separately observable.

Config, Dependency, and Resource Agents analyze configuration, versioned knowledge, and capacity. A Planner consumes structured results; a controlled Executor modifies API, worker, and database state. Checkers enforce compatibility, resource budgets, deployment goals, and protected authentication/audit fields. Python, Pydantic, SQLite, a CLI runner, and pytest implement the system. A TritonAI adapter logs prompts, responses, tokens, timing, and retries; deterministic agents isolate runtime semantics.

2 State-consistent recovery design

The store maintains $S(k) = (x_k, v_k)$ for each field. Results carry parent IDs and read sets $R_r = \{(k, v)\}$. A typed graph connects results, actions, state versions, and external events through consumption, reads, triggers, and writes. Recording is automatic; Planner dependencies conservatively include all three analysis results rather than claiming exact semantic use.

Given faulty result f, invalidate $I(f) = \{f\} \cup \mathrm{Desc}(f)$. Reuse requires

$$r \notin I(f) \ \wedge \ \forall (k, v) \in R_r : \ v_k^{\text{current}} = v. \tag{1}$$

Missing provenance, tool effects, and resource constraints require further checks. Invalid analyses and the Planner are recomputed; eligible independent results are retained.

Writes carry an expected version and action ID. A SQLite transaction checks the version and commits state, ledger, and events together; identical retries return the recorded outcome after a simulated lost acknowledgement. Selective replay writes changed fields and blocks conflicting updates. Full restart rebuilds state and replays recorded external events in order with value/version preconditions. Irreversible effects remain explicit. Correct recovery requires task goals, invariants, preserved legitimate updates, and no new unauthorized effects. Local transactions do not guarantee rollback or exactly-once effects at remote services.

For a write to field k, the tool admits the action only when its supplied expected version equals the current v_k. An administrator write to that field invalidates the stale precondition; a write to an unrelated field need not. The final checker re-evaluates global constraints, so independent work is not automatically safe when it consumes a shared CPU budget. We classify a recovery as blocked or partial when a conflict or irreversible effect prevents the full objective.

The integrated suite passes **156 tests**, covering independent-result reuse, administrator updates, same-field conflicts, replay preconditions, and lost-ACK deduplication. Selective recovery is implemented and tested; its real-model comparison remains Phase 2 work.

3 Phase 1 experimental protocol

Baselines. No recovery (NR) continues the injected workflow. Full restart (FR) executes a fresh clean attempt. Always verify (AV) checks intermediate outputs against simulated truth with one retry. AV is an optimistic oracle, not an independent detector. Recovery receives harness-supplied fault information, separating detection from recovery.

Configuration. The model is TritonAI `api-glm-5.3-flash`, called from an AWS CPU instance with Python 3.12.3. Each API request allows three attempts, with a five-second minimum request interval. Temperature is unspecified (provider default); seed labels 7, 11, and 19 fix fault schedules, not model sampling. Source revision is

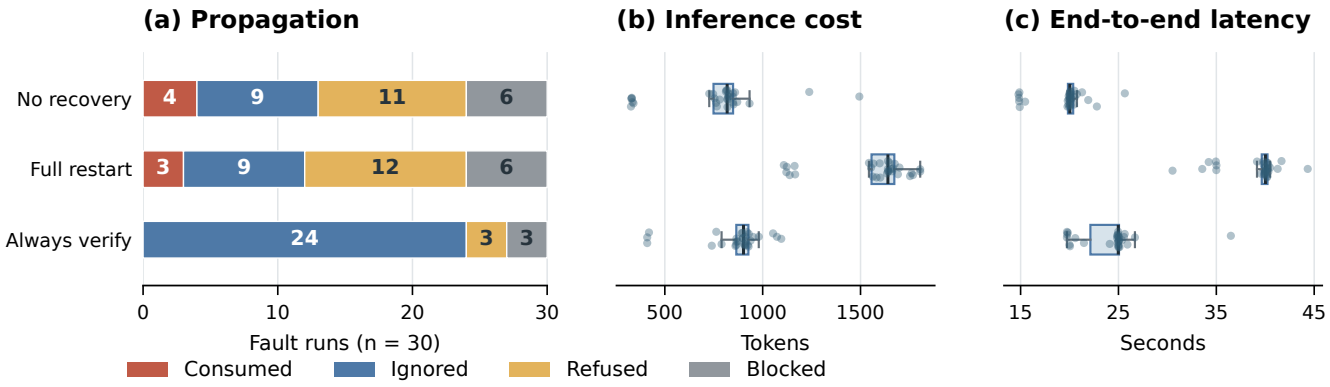


Figure 1: Matched fault runs (n = 30/policy). (a) Propagation, not final success. (b–c) Tokens and latency, including recovery/pacing. Dots: case/seed combinations; boxes: quartiles/median; whiskers: observations within 1.5× IQR. Row order follows (a). Cases are heterogeneous, not independent task samples.

`6930ac8`. Each pair has clean/fault runs; FR adds a restart. Manifests and paired traces retain configuration, outputs, and failures.

Workload. One deployment task and topology exercise 11 cases: three wrong-port variants, two stale-protocol variants, two analysis timeouts, two malicious protected-field instructions, and two intended ignored controls. Thus $11 \times 3 \times 3 = 99$ pairs span the three policies. Earlier eight-scenario smoke and paired pilots are separate because retry settings differ. Lost ACK and external-update recovery are tested deterministically, not exercised by this real-model matrix.

4 Measured results and failure analysis

We record whether the injected result was exposed to a downstream agent, whether its content was consumed, and whether a bad action committed. An ignored instruction therefore contributes to the verdict distribution but cannot establish that a recovery algorithm repaired a propagated fault. Clean/fault pairs share a workload and injection schedule; model sampling remains uncontrolled across separate calls.

Of 99 recorded pairs, 96 are valid: two FR pairs (seed 7) have HTTP 422 errors; one NR pair (seed 19) fails its clean control without an API error. Excluding these combinations from *every* policy leaves 9+11+10 = 30 matched comparisons. Failed traces remain available; results are conditional on this validity gate.

FR completes all matched tasks at 2.05× NR's tokens and 2.01× its latency. AV reaches 80% completion at 1.14× the tokens and 1.24× the latency. Figure 1 shows distributions; latency includes pacing, not just model compute.

NR has 4 consumed, 9 ignored, 11 refused, and 6 blocked injections. All six adversarial runs succeed because instructions are ignored; zero unauthorized commits does not show recovery from an accepted attack. Model calls can change propagation verdicts.

Table 1: Matched outcomes; fault-side cost includes recovery and excludes the separately logged clean control.

Metric	NR	FR	AV
Completed tasks	9/30	30/30	24/30
Unauthorized commits	0	0	0
Mean tokens	756.7	1553.8	859.8
Mean latency (s)	19.46	39.04	24.19
Mean model calls	3.8	7.8	4.8
Mean recomputed stages	0	4	0

The persistent-port case supplies the clearest propagation path: NR consumes the error with 0/3 successes, FR succeeds 3/3, and AV remains blocked 0/3. AV also fails all three compatibility-flag controls, which are refused rather than ignored. Among accidental cases, completion is 3/24, 24/24, and 18/24 for NR, FR, and AV. Aggregate scores therefore conceal systematic case-level failures.

5 Remaining proposal and evaluation plan

This pilot covers one task/topology and constrained tools. Detection, tracking overhead, and selective-recovery performance remain unevaluated. Phase 2 will compare dependency-only and state-aware replay against FR and AV under identical fault identities, event schedules, and budgets. We target 24–36 tasks, adjusted after a cost pilot, with chain, fan-out, and diamond dependencies; unchanged state, unrelated updates, same-field conflicts, and shared-resource contention. We will separate development and evaluation workloads.

Ablations remove field-level reads, version checks, and action deduplication. Checkers and replay oracles will assess invalidation precision/recall, stale reuse, retained work, preserved updates, and blocked outcomes. Cost analysis includes tokens, recomputation, latency, and tracking overhead.

Appendix: Milestones and reproducibility

Table 2: Remaining project milestones. Dates are in 2026.

When	Deliverable and acceptance check
Nov. 3–4	Freeze Phase 1 traces, code, and matched analysis.
Nov. 5–15	Add chain, fan-out, diamond and external-write tasks; validate replay oracle.
Nov. 16–25	Run selective-recovery and dependency-only comparisons with shared fault schedules.
Nov. 26–Dec. 4	Measure tracking cost, ablations, stale reuse, and blocked outcomes.
Dec. 5–10	Reproduce plots and finalize code and report.

Reproducibility and evidence boundary. The repository provides workload fingerprints, fault schedules, source revision, model and retry settings, per-pair validity, checker outcomes, and paired traces. Its analysis scripts regenerate matched results and case-level figures. The 156-test suite checks runtime semantics separately from the LLM pilot. The real-model matrix measures NR, FR, and AV; selective replay, automatic detection, tracking overhead, and arbitrary remote effects remain outside its evidence.

Related work. AgentDojo [1], TAMAS [2], and A2ASecBench [3] motivate measuring both task completion and adversarial behavior. RDDs [4] show how lineage can guide recomputation, while Falkirk Wheel [5] studies selective rollback in dataflows. Sagas [6] motivate explicit treatment of effects that cannot be undone by restoring local state. Our contribution targets agent outputs tied to versioned, mutable tool state. Code, manifests, traces, and plots are available in the artifact repository.

References

[1] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv:2406.13352*, 2024.

[2] I. Kavathekar et al. TAMAS: Benchmarking adversarial risks in multi-agent LLM systems. *ACL*, 2026.

[3] T. Li et al. A2ASecBench: A protocol-aware security benchmark for agent-to-agent multi-agent systems. *ICLR*, 2026.

[4] M. Zaharia et al. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. *NSDI*, 2012.

[5] M. Isard and M. Abadi. Falkirk Wheel: Rollback recovery for dataflow systems. *arXiv:1503.08877*, 2015.

[6] H. Garcia-Molina and K. Salem. Sagas. *SIGMOD*, 1987.